VCAlign: An Effective method for the Removal of Random Delays by Vertical Clustering

Qianmei Wu, Chengdong Xie, Wei Cheng, Fan Zhang

***Abstract*—Random Delay Insertion (RDI) is one of the most investigated types of *hiding* countermeasure. Multifarious approaches to compromise RDI have emerged over the past two decades, with a horizontal perspective considering waveform segment as a unit. Interestingly in this paper, we transform such traditional perspective to a novel vertical one and find that the distribution feature of samples extracted vertically exposures the existence of delays directly, which can be exploited for both RDI detection and measurements alignment. On this basis, we conceive a generic approach called *VCAlign*, leveraging binary classification to align the measurements sample by sample vertically. This approach favors adversaries significantly since it requires no reference trace, no prior knowledge, no profiling stage. As a case study, we further propose a practical paradigm and evaluate it on several RDI-protected implementations on an ARM Cortex-M4 micro-controller. The experimental results demonstrate that VCAlign has a powerful capability to eliminate the delays completely while remaining the encryption-related segments, which can be regarded as an enhanced alignment solution to conquer RDI. We firmly believe that VCAlign is a valid application of instruction effects on physical leakage from the novel vertical perspective that could bring new vitality to the alignment solutions.**

***Index Terms*—Random Delay Insertion, Alignment, Side-channel Leakage, Clustering.**

I. INTRODUCTION

SIDE-CHANNEL Attack (SCA) is nowadays regarded as a non-negligible threat in real world, since it provides another interesting perspective to challenge the security of cryptographic primitives, transferring the attention from traditional black-box attacks in mathematical world to gray-box attacks in physical world. The fundamental idea of SCA is to capture the unavoidable but informative physical leakage during the encryption process running on the embedded devices, and thus disclosure the secret key by leveraging the connection between the collected leakage and the intermediate variables being processed. Such leakage could be processing time [1], power consumption [2], electromagnetic radiation [3] and so on. Due to the validity of SCA and the accordingly potential severe damage, the countermeasures to compromise SCA gain a widespread concern in the past two decades in SCA community. Among multifarious solutions, one of the most investigated and widely applied strategies is "*hiding*" [4].

Hiding conceals the sensitive intermediate variables by cutting down its connection with physical leakage, usually by means of desynchronizing the measurements (captured by attackers) at algorithmic level. To achieve measurements misalignment, a usual method is to cause random displacement in the leaked measurements, which is termed as *Random Delay Countermeasures* (RDC) [5]. Zhang et al. [6] further divides RDC into three types—*Random Propagation Delay* (RPD), *Random Delay Insertion* (RDI) [7] and *Random Clock Jitters* (RCJ) [8]. The former two are software-oriented solutions and the last is implemented in hardware. In particular, RPD emerges naturally if the triggers issued by computer arrive at different times, whilst RCJ leverages various clocks to produce different operating frequencies and thus the measurements are desynchronized by small jitters. In this paper, we are more interested in and concentrate on the investigation of RDI.

RDI inserts dummy operations randomly into the encryption algorithm to cause random displacement among the measurements, thus increasing the attack complexity of adversaries. It is verified that the complexity of Differential Power Attack (DPA) and Correlation Power Attack (CPA) grow linearly or quadratically with the standard deviation of trace displacement in the attack point [5], [9]. In most cases (for both academia and industry), a regular dummy loop is utilized as a delay and the length of delay depends on the number of loops to be executed, which is determined by a pre-set random number. Although delays can be constructed by varying instructions (e.g., using dynamic compilation and *compilettes* [10]), regular loop as delays is still extensively leveraged, since varying instructions bear much more implementation costs. Although RDI is unable to defeat SCA fundamentally, it can increase the attack complexity at a very low cost if implemented with regular loops. It is very easy for a cryptographic designer to place regular loops at program. Hence it is common to combine RDI with other countermeasures like *masking* to further raise attack complexity. In practical scenarios, it is an intractable task for attackers with no prior knowledge to detect and realize the existence of such countermeasure (e.g., especially when combined with other strategies like *masking*). Therefore, although RDI implemented with regular loops seems to be a lazy design, it is an effective and economical selection in practice, which is still extensively utilized in the community

This work was supported in part by National Key R&D Program of China (2023YFB3106800), by National Cryptologic Science Fund of China (2025NCSF02058).

Q. Wu and F. Zhang are with the School of Cyber Science and Technology, Zhejiang University, Hangzhou, 310007, China and with the College of Computer Science and Technology, Zhejiang University, Hangzhou, 310027, China (email: qianmei@zju.edu.cn; fanzhang@zju.edu.cn).

C. Xie is with the College of Mathematics, Sichuan University, Chengdu, 610064, China (email: sanweige@stu.scu.edu.cn).

W. Cheng is with LTCI, Télécom Paris, Institut Polytechnique de Paris, 91120 Palaiseau, France, and also with Secure-IC S.A.S., 75014 Paris, France (e-mail:wei.cheng@telecom-paris.fr).

and industry. In the remainder, RDI specifically refers to the type implemented with regular loops for convenience.

A. Generation methods of RDI

The mainstream generation methods focus on the distribution of the cumulative sum of delays inserted. The simplest method *Plain Uniform Delays* is proposed in CHES 2000 [5], which is a straightforward approach that generates individual delays independently with the length uniformly distributed in the interval $[0, l]$ for $l \in \mathbb{N}$. In 2007, Benoit and Tunstall [11] modified the distribution of an individual delay from the uniform to a pit-shaped one. Instead of generating an individual delay independently, *Floating Mean* (FM) method [12] proposed in CHES 2009 generates the random delays non-independently, significantly improving the variance of the cumulative delay, and is more efficient compared to the methods of both [5] and [11]. In 2010, the authors [13] further analyzed the weakness of FM and devised the *Improved Floating Mean* (IFM) method. Note that FM and IFM methods with regular loops are very common in the implementation of RDI.

B. Attacks against RDI

A naive attack strategy against RDI consists in acquiring and processing more traces to improve the *Signal-to-Noise Ratio* (SNR). In addition, due to the similarity of delay patterns, it is regarded that cross-correlation can effectively break RDI [14]–[16]. Figure 1 exhibits the correlation coefficients between a delay pattern and one trace protected by RDI. From Figure 1, it is shown that a threshold should be set to select the promising beginning point of one delay, which is challenging to decide in practice (especially in noisy scenarios) without prior knowledge and manual interaction. Recently, Neural Networks based attacks are introduced to SCA and confirm much advantage to compromise jitters and random delays [16]–[20]. Those attacks require the adversary to possess a copy of target device, and thus it can profile leakage distribution for successful attacks. However, the copy (or similar) open devices may not be accessible in practice. This is also true with other profiled attacks against RDI, such as methods utilizing opcode matching [21] and Hidden Markov Models [15]. Both of them intended to model the delay-related (resp., encryption-related) instructions (by HW of opcode or Hidden Markov states) and thus distinguish them from encryption-related (resp., delay-related) instructions. Hence these methods require some prior knowledge about the target (e.g., architecture and instruction set of target device, and implementation details of encryption).

Regarding non-profiling attacks, which tend to be more easy to launch in practice, a variety of research literature works emerge in the past two decades. Among which, Frequency Attack (FA) [22], [23] is effective since it is time-invariant, however [24] pointed out that FA cannot succeed when the maximum window size of Discrete Fourier Transform (DFT) is smaller than the length of delays. In addition, the most basic method—*static alignment method* was first proposed in [4], which mainly works in the scenarios of intrinsic jitters. Later, an advanced alignment method—*Elastic Alignment* (EA) method was proposed in [25], which exploits

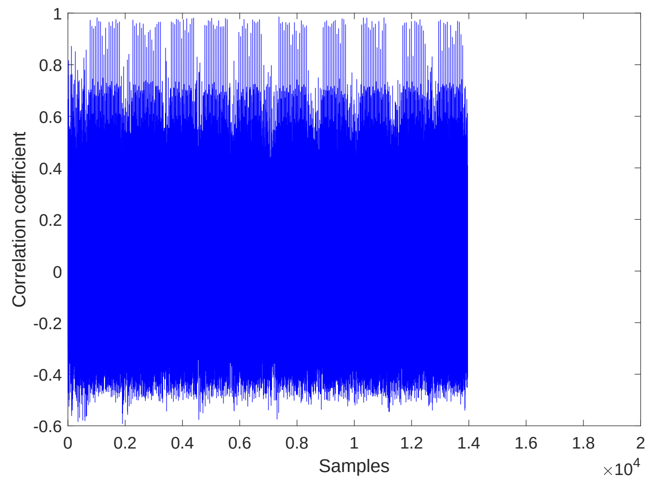


Fig. 1. Cross-correlation between a delay pattern and one protected trace.

fastDWT method [26] to perform nonlinear resampling on the traces. EA is able to match different parts at different offsets, which is well acknowledged by SCA community and widely applied in practice. However, it destroys the trace waveform to some extent due to the fact that it compresses or stretches areas rather than eliminating the delays. In the following, Thiebeauld et al. [27] introduced a new side-channel technique called *scatter* to tackle alignment issue. *scatter* leverages a novel representation and processing method for side-channel measurements. By integrating multiple points—regardless of adjacency—it converts them into a value occurrence distribution that captures leakage effectively. Unlike traditional window-based approaches, *Scatter* maintains accuracy even with non-adjacent point selection, offering greater flexibility and relevance in leakage analysis. Other alignment methods like [28]–[32] are proposed in the past two decades as well. To sum up, there are plenty of attempts and effective approaches to synchronize RDI-protected measurements by aligning the attack point or removing the delays. Unfortunately, there exist more or less some limits to render them to be less effective in practical scenarios, such as high requirements on the attackers (e.g., access to target device, prior knowledge about implementation details). Moreover, it lacks an efficient strategy to detect the existence of random delays countermeasure. The current attacks almost (to the best of our knowledge) assume the adversary who has pre-known the exact countermeasure, which is not realistic in practice.

In this paper, we propose a practically effective method for the removal of random delays. It requires no prior knowledge and no profiling stage. In addition, it has the ability to detect the existence of random delays at the beginning. Interestingly, it provides a novel vertical perspective for measurements synchronization. In detail, we find that most of alignment approaches for RDI rely on a reference (whether it is a waveform segment [25], [29], [30] or a string of HM values [21] or states [15]) and the alignment is achieved by matching the remaining measurements with this reference trace by trace. This is a natural routine for synchronization if considering an individual trace as a unit from a horizontal perspective, as illustrated in Figure 2. However in this paper, we alter the

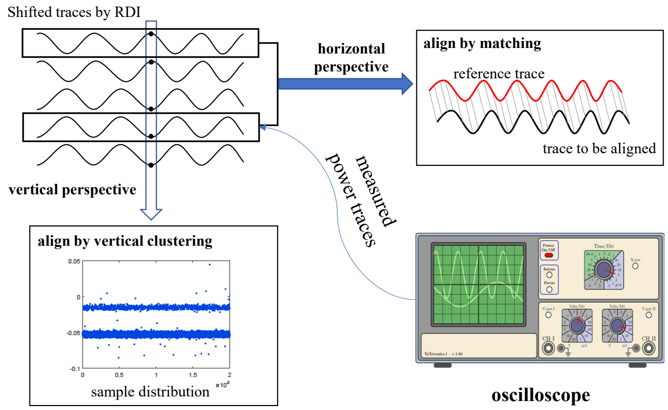


Fig. 2. High-level alignment strategies for horizontal and vertical perspectives.

traditional horizontal perspective to a novel vertical one (also shown in Figure 2) and find that the distribution characterization of samples extracted vertically can be exploited for both RDI detection and measurements synchronization.

C. Contributions

In this paper, we provide **a novel perspective** for measurements synchronization. Transforming the traditional horizontal perspective to a vertical one, the instruction effect on the physical leakage gives its full play: the samples (extracted vertically) involving different instructions present distinguishable leakage distributions. This feature directly leads to **a detection method for random delays**, enabling an adversary with no prior knowledge to quickly check if the collected measurements are protected with random delays at the beginning stage of attack, which is helpful for determination of attack strategy in practical scenarios. Also based on the novel perspective, we conceive **a generic approach for measurements alignment**. This alignment approach utilizes vertical binary classification to eliminate delays, denoted as *Vertical Clustering Alignment* (VCAlign). VCAlign is simple and easy to carry out which requires no reference trace, no profiling phase and no prior knowledge. We should highlight that both detection method and VCAlign are effective applications of instruction impact on physical leakage, which is the nature of SCA but was unfortunately gradually lost sight in the recent research.

Pragmatically, we propose **a practical paradigm of VCAlign** which leverages *Gaussian Mixture Models* (GMMs) clustering method. We showcase an exemplary which develops a specific clustering strategy in line with the feature of GMMs to improve clustering accuracy. And thanks to the enhanced strategy, the computational complexity can be reduced. We **verify the feasibility and effectiveness of VCAlign with GMMs by practical experiments**. We evaluate it on four RDI generation methods, which are summarized and abbreviated in Table I. The experimental results demonstrate that VCAlign with GMMs is able to eliminate the delays effectively regardless of the distribution of the cumulative delay length. It is also compared with EA method and is shown to outperform EA in terms of the alignment effectiveness.

We believe that VCAlign is more able to point out that RDI is vulnerable, since VCAlign captures the exact essence of random delays and can thereby detect and eliminate RDI. It is very recommended to launch VCAlign at the first stage to remove random delays, remaining encryption-involved waveform segments (that have already been synchronized) for side-channel attack, especially when the targeted implementations are protected with other combined countermeasures.

II. Background and Preliminaries

A. Preliminary

We utilize bold lower cases (e.g., $\mathbf{x}$) to represent the sets and $\vec{\mathbf{x}}$ denotes the vectors, where $|\mathbf{x}|$ (resp., $|\vec{\mathbf{x}}|$) defines the size (resp., length) of the set (resp., vector). $\vec{\mathbf{x}}[i]$ represents the i-th coordinate of the vector, and $\vec{\mathbf{x}}[i:j]$ is used to denote a sub-vector which contains the elements $\vec{\mathbf{x}}[k]$ $(i \le k \le j)$ with the same order as in $\vec{\mathbf{x}}$. The function $\max(\mathbf{x})$ (resp., $\min(\mathbf{x})$) returns the maximum (resp., minimum) value of $\mathbf{x}$, while function $\mathrm{ArgTwoMax}(\mathbf{x})$ gives the indices of two maximum elements in $\mathbf{x}$. We denote with $[n]$ the set of integers from 1 to n (both included). The symbol $\cup$, $\cap$ and $\complement$ is used to denote the union, intersection and complement operation on sets, respectively. Let bold capital letters (e.g., $\mathbf{A}^{r\times c}$ with $\mathbf{A}$ in short) represent matrices constructed with r rows and c columns. $\mathbf{A}[i,*]$ (resp., $\mathbf{A}[*,j]$) denotes the i-th row (resp., j-th column) of $\mathbf{A}$, and $\mathbf{A}[i:j,*]$ (resp., $\mathbf{A}[*,i:j]$) denotes the sub-matrix made up of the i-th to j-th rows (resp., columns) of $\mathbf{A}$. We use $\mathbf{A}[\mathbf{x},i:j]$ to represent the matrix made up the row vectors $\vec{\mathbf{y}}[i:j]$ where $\vec{\mathbf{y}}$ is the x-th row of $\mathbf{A}$ $(x \in \mathbf{x})$. Note that we utilize $\mathbf{A}[i:j,*] = \emptyset$ (resp., $\mathbf{A}[*,i:j] = \emptyset$) to indicate that the i-th to the j-th rows (resp., columns) of $\mathbf{A}$ are deleted. All indices in this paper start from 1 instead of 0.

Considering an RDI-protected implementation, an individual delay can be inserted at multiple points of the program with varying length. To avoid ambiguity, we use "m delays" to represent the delays that are placed at m different points of the program, while "n-length delay" denotes one individual delay that has n number of loops (or saying its length is n). And we use D^i to represent the delay that is placed at the i-th point, while D_i denotes the i-th loop of one individual delay. For example, D_1^1 denotes the first loop of the first delay inserted at the program.

B. Wavelet Analysis

Wavelet Transform ($\mathbb{WT}$) is regarded as an enhanced tool for signal processing due to its powerful ability of multi-resolution analysis. As illustrated in Equation 2, $\mathbb{WT}$ is conducted by an integration over the product of target signal $f(t)$ and wavelet basis function ψ (shown in Equation 1), where scaling s indicates the expansion and contraction of wavelet and shift u depicts the displacement of wavelet. Hence, with dynamic changing scaling s and shift u, the time-domain signal $f(t)$ shall be analyzed at various resolutions.

$$\psi_{s,u}(t) = \frac{1}{\sqrt{s}}\psi(\frac{t-u}{s}) \tag{1}$$

$$\mathbb{WT}(s,u) = \frac{1}{\sqrt{s}}\int_{-\infty}^{+\infty} f(t) * \psi(\frac{t-u}{s})dt \tag{2}$$

TABLE I
FOUR GENERATION METHODS OF RDI

Abbreviation	References	Methods	Comments
RDI-PU	Clavier *et al.* [5]	Plain Uniform (PU)	The length of an individual delay follows a uniform distribution
RDI-BT	Tunstall *et al.* [11]	Method of Benoit and Tunstall (BT)	The distribution of the length of an individual delay is modified from the uniform to a pit-shaped one
RDI-FM	Coron *et al.* [12]	Floating Mean Method (FM)	Implement the random delays non-independently
RDI-IFM	Coron *et al.* [13]	Improved Floating Mean Method (IFM)	Improve the granularity of random delays generated by FM

There are two types of wavelet transforms: Continuous Wavelet Transform ($\mathbb{CWT}$) and Discrete Wavelet Transform ($\mathbb{DWT}$). Since the physical leakage is usually concretized as discrete samples collected by a digital oscilloscope in SCA community, $\mathbb{DWT}$ is utilized as the main technique for trace processing. $\mathbb{DWT}$ can be called as wavelet decomposition. As illustrated in Figure 3, the wavelet decomposition of a signal is composed of two components: filtering and down-sampling. Filtering is conducted by both a Low Pass Filter (LPF) and a High Pass Filter (HPF). Hence, signals are accordingly divided into low-frequency signals and high-frequency ones. After filtering, those filtered signals should be down-sampled by two to remove redundant information. The process composed of both filtering and down-sampling is called one level of wavelet decomposition. However, the low-frequency signals can be continued to perform the next level of wavelet decomposition. Therefore, signal $f(t)$ will be continuously decomposed to two parts in wavelet domain: the low-frequency ones called *approximation wavelet coefficients* (cA_i) and the high-frequency ones called *detail wavelet coefficients* (cD_i), where i implies the decomposition level. For example, if a three-level decomposition is applied to a signal, cD_1, cD_2, cD_3, cA_3 will be eventually derived. In other words, the time-domain signal is refined into four components at different frequency bands. It is worth noting that, to conduct a wavelet decomposition, appropriate wavelet basis function and decomposition level should be determined. In general, wavelet basis functions can be classified into different families, such as Daubechies (DbN), Mexican Hat (Mexh), Morlet (Morl), Meyer (Meyr), Symmlets (SymN) and Coiflets (CoifN) [33]. The letter "N" in DbN, SymN and CoifN indicates the order of the wavelet basis function. For example, Db1 has the order of one.

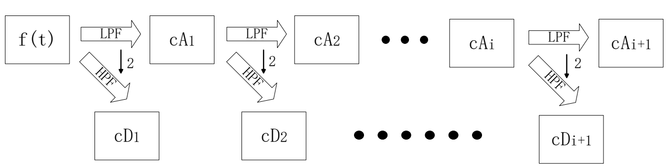


Fig. 3. Discrete Wavelet Transform ($\mathbb{DWT}$) at multiple levels.

III. Vertical Clustering Alignment

In this section, we first introduce our threat model and the challenges. In the following, we elaborate the leakage distribution characterization of samples extracted vertically from measurements and introduce the detection method for random delays. Based on the leakage distribution feature from vertical perspective, the generic VCAlign approach is proposed and detailed. In addition, we also analyze the feasibility and potential advantage of VCAlign in wavelet domain.

A. Threat Model and Challenges

In this work, we consider a threat model where the target cryptographic implementation is protected using RDI, achieved by inserting regular dummy loops whose iteration count determines the length of each delay. The adversary possesses no prior knowledge regarding the RDI implementation—such as the quantity, positions, or lengths of the inserted delays—and may not even be aware that such a countermeasure is deployed. The core challenge is to accurately detect as well as remove these delays, and align leakage measurements without relying on any prior knowledge about the target, template modeling, or manually selected reference waveform—common prerequisites in existing alignment techniques. This scenario calls for a blind and adaptive alignment strategy capable of operating under noisy conditions while remaining efficient and applicable in real-world adversarial settings.

B. Leakage Distribution from Vertical Perspective

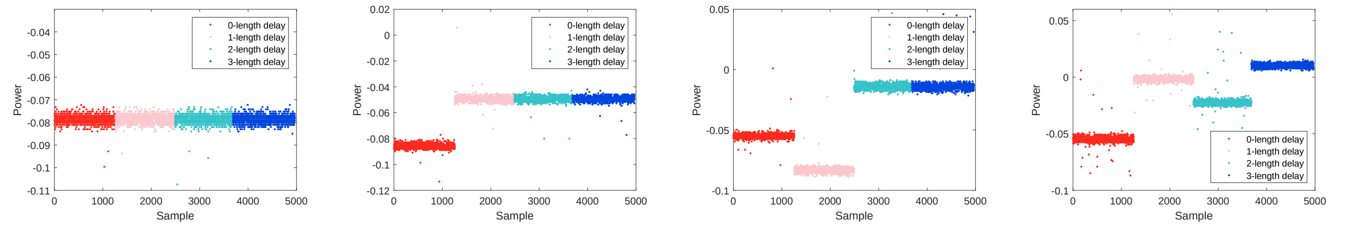
Recall that each point of a leakage (e.g., power consumption) trace can be modeled as the sum of an operation-dependent component P_{op}, a data-dependent component P_{data}, noise component P_{noise} and a constant component P_{const} [4], as shown in Equation 3.

$$P_{total} = P_{op} + P_{data} + P_{noise} + P_{const}. \tag{3}$$

Now we consider a fixed moment of time and thus P_{const} keeps consistent. As claimed in [4], P_{noise} follows a normal distribution with mean μ and variance σ, while P_{data} can be usually approximated by a Hamming Weight (HW) model. Thus for a specific operation (with the same P_{op}), it can be inferred that P_{total} follows a *Gaussian Mixture Model*, with its *Expectation* evaluated as in Equation 4, where $P_{hw=4}$ indicates the P_{data} whose HW of the corresponding data equals to 4.

$$E(P_{total}) = P_{op} + P_{hw=4} + \mu + P_{const}. \tag{4}$$

Hence $E(P_{total})$, denoted as *center*, is determined by the operation-dependent component P_{op}. At a specific moment of time, the samples will have n different leakage distributions (consistent *Gaussian Mixture* distributions with n different *centers*) if they involve n instructions. Accordingly, it can be inferred that if the difference between P_{op} exceeds the whole value range of P_{data} (from $P_{hw=0}$ to $P_{hw=8}$), the samples should present distinguishable clusters, as shown in Figure 4b,



(a) $j \in [P_0, P_1)$ (b) $j \in [P_1, P_2)$ (c) $j \in [P_2, P_3)$ (d) $j \in [P_3, P_4)$

Fig. 4. Leakage distributions of samples $\mathbf{T}[*, j]$ where $\mathbf{T}$ denotes a sub-set of measurements from RDI-PU whose implementation details will be provided in Section V. Note that those points in one sub-figure are actually sampled at the same moment of time, and x-axis represents the meaningless index of samples. The "x-length delay" with $0 \leq x \leq 3$ and P_i with $0 \leq i \leq 4$ will be explained below in Section III-C.

Figure 4c and Figure 4d, where the points involve 2, 3 and 4 different instructions, respectively, with Figure 4a depicting the samples distribution when involving the same instruction.

Recall that RDI inserts delays randomly at the calculation sequence and thus the essence is that it diversifies the instruction types of samples at the same moment of time, rendering P_{op} to be informative. Accordingly, the explicit benefit is to allow the adversary without any prior knowledge to quickly check if there exists random delays for protection. Besides, other countermeasures which enable the instructions flow vary with different queries, such as *shuffling*, can be detected as well. The detection approach is to search the samples from the beginning of measurements (or with specific strategy) and check if there exists distinct distributions. Certainly, countermeasures can also be detected by comparing the waveform patterns, however searching for the difference among multifarious patterns is indeed a tough work, especially in a case usually with numerous and long curves. The advantage of the former detection method lies in the novel vertical perspective. Since the samples are extracted vertically, it covers all the traces at a time, while the latter requires to compare difference trace by trace (and segment by segment as one trace is usually very long) from a horizontal perspective.

C. Leakage Distribution of RDI

Given a set of RDI-protected measurements $\mathbf{T}^{r \times c}$ with $\mathbf{T}[i, *]$ denoting the i-th trace and $\mathbf{T}[*, j]$ representing the points sampled at time j, we simply suppose that only one delay with n as its largest length is inserted. And we denote P_i ($1 \leq i \leq n$) as the position where D_i is placed, with P_0 a special case representing the beginning point of $\mathbf{T}$. From a global point of view, the measurements $\mathbf{T}$ can be divided into $n+1$ groups according to its delay length and we use $\mathbf{T}^k$ to represent the measurements group whose delay length is k ($0 \leq k \leq n$). From a dynamic perspective, we extract samples $\mathbf{T}[*, j]$ vertically with j starting from P_0, the samples distributions from different $\mathbf{T}^k$ should keep consistent until j moves to P_1, as shown in Figure 4a where "x-length delay" indicates samples from $\mathbf{T}^x$. Certainly, those traces segments $\mathbf{T}[*, P_0 : (P_1 - 1)]$ are already synchronized[1]. When j is sampled at $[P_1, P_2)$, the samples $\mathbf{T}[*, j]$ should split out into two distinguishable clusters as shown in Figure 4b, since samples from $\mathbf{T}^0$ still undergo the normal encryption operations while the other samples from $\mathbf{T}^k$ ($k \geq 1$) involve the insertion of D_1. Note that the waveform segments before and during the insertion of D_i keep consistent among the groups $\mathbf{T}^k$ with $k \geq i$. This is because D_i indicates that the regular loop repeats for the i-th time. And the repeated loops for i times are the same for $\mathbf{T}^k$ ($k \geq i$). Continuing to move j to the range $[P_2, P_3)$, samples $\mathbf{T}[*, j]$ should present three distinct distributions (as shown in Figure 4c) since $\mathbf{T}^0$ and $\mathbf{T}^1$ are performing encryption operations at different progress (precisely $\mathbf{T}^0$ is ahead of $\mathbf{T}^1$ by one length delay), while $\mathbf{T}^k$ ($k \geq 2$) undergo the insertion of D_2. Continuously, it follows the same rule that when $j \in [P_i, P_{i+1})$, the associated samples $\mathbf{T}[*, j]$ will present $i+1$ different leakage distributions. Thus ultimately there will exist $n+1$ clusters among $\mathbf{T}[*, j]$ with $j \in [P_n, c]$ after the insertion of D_n.

Such leakage feature is an ideal case with only one delay considered. In practice, multiple short delays will be placed at different points of program. Hence if measurements $\mathbf{T}$ have m delays inserted with largest length as n, it can actually possess m^n groups of sub-traces according to the length of each delay and thus leakage distribution becomes very complicated. Actually even if only one delay is inserted, sometimes the $i+1$ distributions during the insertion of D_i may not be that distinguishable due to the noise and special situations—Some samples involve the same instructions even if they are from different groups, or some samples involve the similar operations of the underlying micro-controller even if their corresponding instructions are different. However we should highlight that, whatever how multiple delays will be inserted, there always exists P_1 where D_1^1 is placed and thus two different samples distributions will occur. Those two different distributions are usually discriminated since P_1 usually involves branch instructions for delay insertion, which causes remarkable difference in the samples distributions.

D. Vertical Clustering Alignment

As illustrated in Figure 2, we intend to synchronize the measurements in a vertical way, instead of matching the traces with a reference from a horizontal perspective. It is analyzed in Section III-C that the samples $\mathbf{T}[*, j]$ extracted vertically from measurements $\mathbf{T}$ will break into several clusters due to the insertion of random delays. Hence a natural idea for synchronization is to carry out clustering algorithm on those

[1]In practice, minor jitters are usually present (e.g., due to various triggers delay). However in this paper, we disregard these small jitters and consider the measurements without any introduced program-level delay as "synchronized".

samples $\mathbf{T}[*, j]$ to identify which involve the delay operations and thus the alignment can be achieved by removing those samples related to delays. By this way, the measurements can be aligned sample by sample (position by position) with all traces at a time rather than traditionally trace by trace, thus promisingly easing the computational overhead.

Given a set of samples $\mathbf{T}[*, j]$ and suppose that the samples break into n clusters $\mathbf{c}^i$ $(1 \leq i \leq n)$ due to the insertion of random delays. As analyzed in Section III-C, those n clusters may originate from the encryption operations or correspond to random delays. For the sake of convenience, we denote $\mathbf{c}^e$ (resp., $\mathbf{c}^d$) as the clusters corresponding to encryption operations (resp., random delays). An adversary should discriminate $\mathbf{c}^d$ from n clusters and eliminate them. However, the difference among those n clusters only lies in the *centers*, and it is thereby a tough task for attackers with no prior knowledge (e.g., about the specific instructions that make up delays) to identify $\mathbf{c}^d$ without any profiling stage (profiling the values of *centers* for instructions). Moreover, with an increasing n, the clustering accuracy will decrease due to the addition of noise, the reduction in sample amounts for each cluster and the amplified possibility of special situations occurring.

Facing with it, we apply the *divide-and-conquer* principle of SCA and accordingly conceive an feasible alignment approach *Vertical Clustering Alignment* (VCAlign), leveraging the instruction effects on leakage. The basic idea is to remove the delays one by one and eliminate an individual delay one loop by one loop. The only requirement for VCAlign is that the measurements $\mathbf{T}$ should begin before the insertion of an individual delay (which is easy for adversaries to access in practice). Since beginning before the insertion of delays, the samples $\mathbf{T}[*, j]$ extracted vertically at the beginning should share the same distribution. As j moves backwards to the position where D^1 is placed, the samples $\mathbf{T}[*, j]$ split out into two distinguishable clusters, which will stay during $j \in [P_1, P_2)$. Then a clustering algorithm can be conducted on the samples $\mathbf{T}[*, j]$ with $j \in [P_1, P_2)$ to perform a binary classification. Consequently, the cluster of smaller (resp., larger) size is exactly $\mathbf{c}^e$ (resp., $\mathbf{c}^d$). This is determined by the essential goal of RDI[2]. After the identification of delays cluster, the samples $\mathbf{T}[*, j]$ belonging to $\mathbf{c}^d$ shall be eliminated, resulting in that D^1 of group $\mathbf{T}^1$ has been fully removed and D^1_1 of groups $\mathbf{T}^k$ with $k > 1$ is reduced. Therefore, the measurements in groups $\mathbf{T}^0$ and $\mathbf{T}^1$ are already synchronized before the insertion of the second individual delay D^2, and the length of D^1 in groups $\mathbf{T}^k$ with $k > 1$ has been reduced by one.

To avoid ambiguity, we update the new measurements as $\mathbf{T}_{\text{new}}$, and its group $\mathbf{T}^0_{\text{new}}$ actually contains the measurements of both $\mathbf{T}^0$ and $\mathbf{T}^1$. Besides, its largest length of the first delay reduces to $n - 1$ and thus it only has n groups now. Notably, the samples $\mathbf{T}_{\text{new}}[*, j]$ with $j \in [P_1, P_2)$ still possess two distinguishable clusters since the group $\mathbf{T}^0_{\text{new}}$ undergoes the encryption operations, while $\mathbf{T}^k_{\text{new}}$ $(k > 0)$ involves one length delay part (the D^1_2 of the measurements $\mathbf{T}^k$ with $k > 1$). Such procedure is illustrated in Figure 5 with groups $\mathbf{T}^k$ simplified as exemplary traces t_k and supposing $n = 3$. Still a binary classification conducted on $\mathbf{T}_{\text{new}}[*, j]$ with $j \in [P_1, P_2)$, the cluster $\mathbf{c}^e_{\text{new}}$ can be identified by checking if most of the samples in $\mathbf{c}^e$ are included. In this way, $\mathbf{c}^d_{\text{new}}$ can be discriminated and eliminated, and thus D^1_2 of measurements $\mathbf{T}$ can be removed. In other words, the measurements of $\mathbf{T}^0$, $\mathbf{T}^1$ and $\mathbf{T}^2$ are now well synchronized before the insertion of D^2. Following the same rule, the last D^1_n of D^1 in $\mathbf{T}^n$ will ultimately be eliminated and thus D^1 are completely removed from $\mathbf{T}$. The same procedure can be conducted on the remaining delays D^i $(1 < i \leq m)$. After the elimination of delays one by one, the measurements are well synchronized. We illustrate the approach VCAlign step by step below:

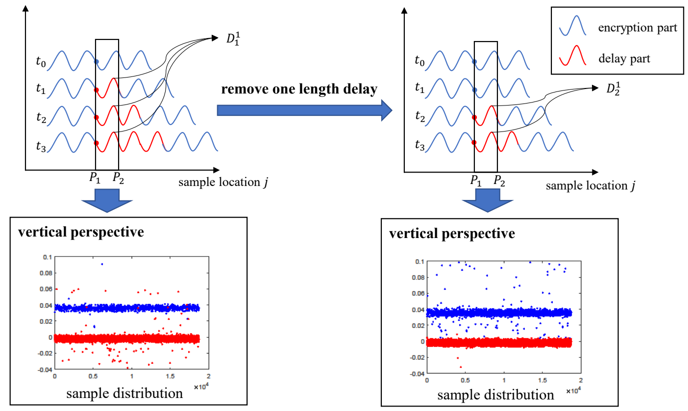


Fig. 5. Simplified illustration for the removal of one length delay by VCAlign.

Step 1. Move j from the beginning point P_{start} and check if the samples $\mathbf{T}[*, j]$ present two distinguishable distributions. If it satisfies, jump to Step 2, otherwise continue to move j to the next one. The approach ends if j arrives at the end of the measurements.

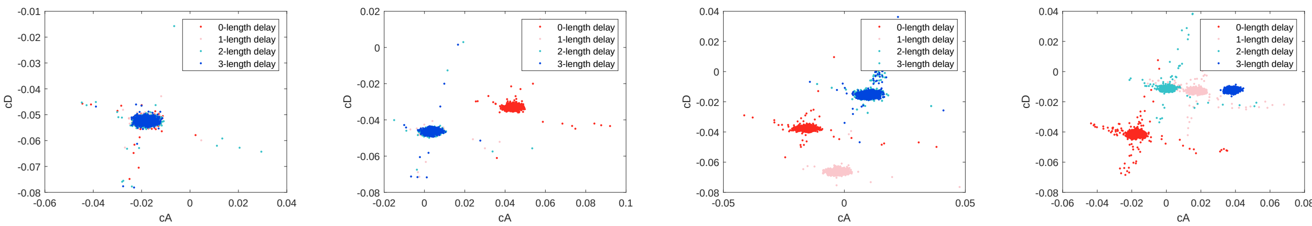
Step 2. Set the point where two distinguishable clusters first emerges as P_1 and continue to move j to find the point P_2 where firstly three distinguishable distributions occurs.

Step 3. Perform binary classification on the samples $\mathbf{T}[*, j]$ with $j \in [P_1, P_2)$ and remove the samples belonging to cluster $\mathbf{c}^d$. The identification method for $\mathbf{c}^d$ is detailed above.

Step 4. Execute Step 3 until there only exist one distribution in samples $\mathbf{T}[*, j]$ during the range $j \in [P_1, P_2)$. If that becomes true, set $P_{\text{start}} = P_1$ and jump to Step 1 again for the removal of the next delay.

By this way, VCAlign is able to eliminate delays one by one utilizing binary classification, remaining the normal encryption part and thus the measurements are synchronized. Note that VCAlign has a very low requirement for adversary—the measurements should begin before the insertion of an individual delay. We should highlight that VCAlign is a generic procedure which requires specific clustering algorithm and the matched clustering strategy, and thus it holds boundless potential for delays elimination. We firmly believe that VCAlign with a novel vertical perspective provides a fresh and powerful alternative for delays elimination, which grabs the exact essence of RDI and brings new vitality to exploitation of the instruction effects in side-channel leakage.

[2]If the group with no delay is larger than those with delays, the validity of RDI will be greatly reduced.



(a) $j \in [P_0, P_1)$ (b) $j \in [P_1, P_2)$ (c) $j \in [P_2, P_3)$ (d) $j \in [P_3, P_4)$

Fig. 6. Leakage distributions of $\mathbf{v}_w = [cA_1, cD_1]$ in wavelet domain.

E. VCAlign in Wavelet Domain

VCAlign is based on the fact that samples involving different instructions present distinguishable leakage distributions. If we consider the time-domain sample as a kind of feature expression of the corresponding instruction, it is natural to think of other ways to represent the instructions. We denote such expression as a feature vector $\mathbf{v}$ and thus time-domain sample is an vector $\mathbf{v}_t$ with $|\mathbf{v}_t| = 1$. In this direction, wavelet analysis can be utilized to produce a new feature vector $\mathbf{v}_w$. Recall that wavelet decomposition has the capacity to transform the time-domain sample into wavelet-domain coefficients, refining the information contained in time-domain sample into various components within a certain frequency range. Specifically as detailed in Section II-B, an l-level wavelet decomposition will transform the signal f into $l+1$ wavelet coefficients, that is $f = cD_1 + \cdots + cD_l + cA_l$. In other words, one-dimensional feature vector $\mathbf{v}_t$ is converted to the $(l+1)$-dimensional $\mathbf{v}_w$ with $|\mathbf{v}_w| = l+1$ if l-level wavelet decomposition is conducted.

To verify the feasibility of $\mathbf{v}_w$, we perform wavelet decomposition with wavelet basis function Db2 and 1-level decomposition level on the samples $\mathbf{T}[*, j]$ from sub-traces of RDI-PU that covers the delay length from 0 to 3 (as in Figure 4). Figure 6 plots the leakage distributions of $\mathbf{v}_w = [cA_1, cD_1]$ in wavelet domain corresponding to the sub-figures of Figure 4 one by one. It can be indicated that the wavelet-domain feature vector can be exploited for VCAlign as well. We believe that if with proper wavelet decomposition, the leakage distributions of wavelet-domain $\mathbf{v}_w$ could be more informative than $\mathbf{v}_t$ due to the increase in dimension and the noise reduction. In addition, the advantage of wavelet-domain VCAlign also lies in the decrease of the points amount to be searched. Since wavelet decomposition performs down-sampling by 2 at each level, the samples amount c of measurements $\mathbf{T}^{r\times c}$ will halve after each-level decomposition, which shall significantly ease the computational overhead of VCAlign. However, the shortcoming is that wavelet decomposition itself requires high computational complexity, although it can be reduced by the use of block wavelet [29] which costs $\mathcal{O}(1)$.

IV. Practical Paradigm of VCAlign

As indicated in Section III, VCAlign requires an effective clustering algorithm for binary classification. In this section, we analyze the clustering performance of GMMs and provides a practical paradigm of VCAlign, which demonstrates the feasibility of VCAlign.

TABLE II
Average Clustering Accuracy of GMMs

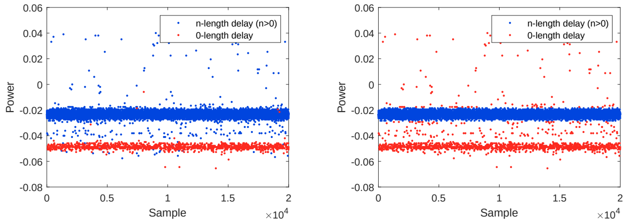
	RDI-PU	RDI-BT	RDI-FM	RDI-IFM
Naive strategy	73.40%	78.51%	80.54%	81.72%
Improved strategy	100.00%	100.00%	99.64%	99.99%

A. Clustering Analysis of GMMs

Implied by the procedure of VCAlign detailed in Section III-D, the accuracy of binary classification on samples $\mathbf{T}[*, j]$ during $j \in [P_1, P_2)$ plays a significant role in VCAlign. It is worth noting that the indices of samples $\mathbf{c}^e \subset \mathbf{T}[*, j]$ (resp., $\mathbf{c}^d \subset \mathbf{T}[*, j]$) involving the encryption (resp., delay) operations keep identical for all $j \in [P_1, P_2)$. Hence on the one side, once the correct indices for each cluster are obtained, the adversary is able to eliminate the whole D_i with no need of exhaustion on $[P_1, P_2)$. On the other side, the distribution information of samples $\mathbf{T}[*, j]$ for all $j \in [P_1, P_2)$ can be combined and jointly leveraged to perform a more accurate binary classification.

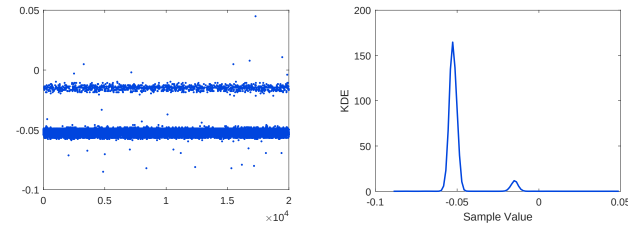
As analyzed in Section III-B, the leakage distribution of samples extracted from a fixed moment of time follows the *Gaussian Mixture* distribution, thus we select GMMs with *Expectation Maximization* optimization algorithm for clustering. We evaluate its accuracy on four RDI-protected implementations summarized in Table I. As a simple test, we carry out binary classification on the samples $\mathbf{T}[*, j]$ for each $j \in [P_1, P_2)$. The average clustering accuracy results for RDI-PU, RDI-BT, RDI-FM and RDI-IFM are 73.40%, 78.51%, 80.54% and 81.72%, respectively, as shown in the second row of Table II. Hence for each protected implementation, there exit some samples which can not be fully classified into correct clusters. This might be owed to that the clustering algorithm is not sufficiently effective or the samples distributions are not distinguishable. However, either situation will have a non-negligible negative impact on the alignment of VCAlign, which definitely requires improvement.

To improve the accuracy, we conduct an in-depth analysis on the clustering performance of GMMs. Figure 7 illustrates the distributions of the samples $\mathbf{T}[*, j]$ with j randomly selected in $[P_1, P_2)$ from RDI-PU implementation. It can be indicated in Figure 7a that some points of the two clusters overlap each other due to the noise. Facing this situation, GMMs clustering method tends to classify points with high-density as a group and then the remaining points become a group naturally no matter how scattered they are. Precisely in Figure 7a, the blue points $\mathbf{c}^b$ belonging to $\mathbf{T}^k$ $(k > 0)$ group could be roughly divided into two sets—dense points set $\mathbf{c}^b_{\text{dense}}$ and scattered



(a) Original clusters (b) Clusters by GMMs method

Fig. 7. Clustering performance of GMMs clustering method.



(a) Leakage distribution (b) KDE result

Fig. 8. Samples $\mathbf{T}[*, j]$ distribution (left) of RDI-PU with j randomly selected in $[P_1, P_2)$ and its corresponding KDE result (right), calculated by ksdensity function of Matlab toolbox.

points set $\mathbf{c}^b_{\text{scatter}}$. Since $\mathbf{c}^b$ is far more than the red points $\mathbf{c}^r$ from $\mathbf{T}^0$, the density of $\mathbf{c}^b_{\text{dense}}$ is higher than that of $\mathbf{c}^r$. Hence as indicated in Figure 7b, GMMs clustering method classifies the $\mathbf{c}^b_{\text{dense}}$ as a group $\mathbf{g}^l = \mathbf{c}^b_{\text{dense}}$ and combines the $\mathbf{c}^b_{\text{scatter}}$ and $\mathbf{c}^r$ as another group $\mathbf{g}^s = \mathbf{c}^b_{\text{scatter}} \cup \mathbf{c}^r$. Now we use $\mathbf{c}^l$ (resp., $\mathbf{c}^s$) to represent the cluster of larger (resp., smaller) size. If two clustering operations are carried out on samples $\mathbf{T}[*, j_1]$ and $\mathbf{T}[*, j_2]$, respectively, for $j_1, j_2 \in [P_1, P_2)$ and $j_1 \neq j_2$, four groups can be attained: $\mathbf{g}^s_1 = \mathbf{c}^l_{\text{scatter}} \cup \mathbf{c}^s$, $\mathbf{g}^l_1 = \mathbf{c}^l_{\text{dense}}$ for the first clustering and $\mathbf{g}^s_2 = \mathbf{c}^{l'}_{\text{scatter}} \cup \mathbf{c}^s$, $\mathbf{g}^l_2 = \mathbf{c}^{l'}_{\text{dense}}$ for the second one. Recall that $\mathbf{c}^s$ and $\mathbf{c}^l$ are the same for both $\mathbf{T}[*, j_1]$ and $\mathbf{T}[*, j_2]$ if without introducing any noise. Hence $\mathbf{c}^e$ can be obtained by $\mathbf{c}^e = \mathbf{g}^s$ with $\mathbf{g}^s$ calculated by Equation 5 and $\mathbf{c}^d$ can be attained by $\mathbf{c}^d = \complement_{[r]}\mathbf{c}^e$.

$$\begin{aligned} \mathbf{g}^s &= \mathbf{g}^s_1 \cap \mathbf{g}^s_2 = (\mathbf{c}^l_{\text{scatter}} \cup \mathbf{c}^s) \cap (\mathbf{c}^{l'}_{\text{scatter}} \cup \mathbf{c}^s) \\ &= \mathbf{c}^s \cup (\mathbf{c}^l_{\text{scatter}} \cap \mathbf{c}^{l'}_{\text{scatter}}) \end{aligned} \tag{5}$$

Equation 5 indicates that $\mathbf{g}^s = \mathbf{c}^s$ can be perfectly obtained if $\mathbf{c}^l_{\text{scatter}} \cap \mathbf{c}^{l'}_{\text{scatter}} = \emptyset$. This stands a good chance since the scattered samples from $\mathbf{c}^l$ usually take a relatively small size (compared to the dense points) and are randomly distributed. Note that we give an ideal assumption before that $\mathbf{g}^s_1$ (resp. $\mathbf{g}^s_2$) classified by GMMs fully contains the smaller set $\mathbf{c}^s$. Therefore, in order to obtain $\mathbf{c}^e$ correctly, it is required for those two clustering operations that enable $\mathbf{g}^s$ include as many elements as possible in $\mathbf{c}^s$ and make the size of $\mathbf{c}^l_{\text{scatter}}$ smaller. To this end, we conceive an *score* function to select two optimal positions j_1 and j_2 during $[P_1, P_2)$ for those two clustering operations. For each set of samples $\mathbf{T}[*, j]$ with $j \in [P_1, P_2)$, we calculate *Kernel Density Estimation* (KDE) based on a normal kernel function first to obtain the probability density estimate evaluated at equally-spaced points. Since the samples $\mathbf{T}[*, j]$ have two distinguishable clusters, two remarkable peaks will occur in the estimation, as in Figure 8b. Three principles are developed for samples positions selection: 1) the distance D between those two peaks should be as large as possible, indicating that the corresponding clusters are sufficiently distinct and fewer points from those clusters will overlap each other with less possibility; 2) the width w_h of higher peak and w_l of lower peak should be narrow, and thus the clusters are compact; 3) the proportion p of the area of those two peaks should be large enough and accordingly the size of the scattered (noisy) points is small. The significance decreases in sequence. The score s is calculated for each set of samples $\mathbf{T}[*, j]$ with $j \in [P_1, P_2)$ as in Equation 6, where α_i $(1 \leq i \leq 4)$ are coefficients. And the optimal positions are exactly j_1 and j_2 with the highest *scores*. To adapt with the proportion p, the distance D and the width w_h, w_l should be normalized by the sample value space.

$$s = \frac{\alpha_1 D}{\alpha_2 w_h + \alpha_3 w_l + \alpha_4 (1 - p)} \tag{6}$$

If the dimension of feature vector for clustering increases to L (e.g., transforming from time-domain sample to L wavelet-domain coefficients), KDE should be calculated for points in each dimension and thus L estimation results will be attained. Therefore, the *score* function in Equation 6 can be extended as shown in Equation 7.

$$s = \frac{\alpha_1 \prod_{i=1}^{L} D_i}{\alpha_2 \sum_{i=1}^{L} w_{h,i} + \alpha_3 \sum_{i=1}^{L} w_{l,i} + \alpha_4 \sum_{i=1}^{L} (1 - p_i)} \tag{7}$$

Thanks to the improvement of clustering strategy, we obtain the clustering accuracy results 100%, 100%, 99.64% and 99.99% on the same samples sets $\mathbf{T}[*, j]$ with $j \in [P_1, P_2)$ for each implementation protected by RDI-PU, RDI-BT, RDI-FM and RDI-IFM, respectively, as recorded in the third row of Table II. Note that the clustering accuracy stays the same for samples $\mathbf{T}[*, j]$ for each $j \in [P_1, P_2)$. The small classification error in RDI-FM and RDI-IFM is because the proportion of $\mathbf{T}^0$ is too small due to the essential design of FM and IFM generation methods. This case will be ameliorated since the proportion of $\mathbf{c}^e$ will grow with the procedure of VCAlign. Note that this clustering strategy customized for GMMs clustering method can not only apply in the removal of D_1, but also the elimination of D_i $(1 \leq i \leq n)$. This improvement is an example to combine the information of some sets $\mathbf{T}[*, j]$ for various $j \in [P_1, P_2)$, attaining more accurate clustering results. As is shown above, with the ultimate goal of synchronization, the clustering strategy during $[P_1, P_2)$ for VCAlign can be tuned and improved according to the feature of the selected clustering method. Admittedly, there are other excellent clustering algorithms, such as some approaches based on Non-parametric Estimation [34]. We do not keep delving deeper in the search for optimal clustering method in this paper and leave it as a future work.

B. VCAlign with GMMs

In this section, we follow VCAlign approach and propose a practical paradigm utilizing the customized clustering strategy specific for GMMs clustering method (introduced in Section IV-A). Recall in Section III-D, VCAlign synchronizes the measurements by eliminating the delays one by one and it requires specific binary classification method for removal of

TABLE III
ERROR SITUATIONS AFTER VERTICAL CLUSTERING.

	case A	case B
$\mathbf{c}^s = \mathbf{c}^e$	$\mathbf{c}^e_{g^d}$	$\mathbf{c}^d_{g^e}$
$\mathbf{c}^s = \mathbf{c}^d$	$\mathbf{c}^d_{g^e}$	$\mathbf{c}^e_{g^d}$

each delay loop by loop, which is left for instantiating. Now we feed VCAlign approach with GMMs clustering method and illustrate this paradigm in Algorithm 1. In Algorithm 1, function "getClusterAmount" returns the clusters amount of the input samples $\mathbf{T}[*, j]$, which can be achieved by counting the number of peaks in KDE estimation, while function "getOptimalPositions" outputs the optimal j_1 and j_2 during $[P_1, P_2)$ for two clustering operations, as detailed in Section IV-A. Function "Choose" provides the cluster related to the encryption operations and the judgment method has been detailed in Section III-D. Note that notations like $I_{\mathbf{c}}$ indicate the indices of elements which belong to set $\mathbf{c}$.

Algorithm 1 VCAlign with GMMs

Require: Raw measurements $\mathbf{T}^{r \times c}$
Ensure: Aligned measurements $\mathbf{T}_{align}$

1: $j = 1$
2: **while** $j \leq c$ **do**
3:   **while** $\mathrm{getClusterAmount}(\mathbf{T}[*, j]) < 2$ **do**
4:     $j = j + 1$
5:   **end while**
6:   $P_1 = j$
7:   **while** $\mathrm{getClusterAmount}(\mathbf{T}[*, j]) < 3$ **do**
8:     $j = j + 1$
9:   **end while**
10:   $P_2 = j$
11:   $[j_1, j_2] = \mathrm{getOptimalPositions}(\mathbf{T}, P_1, P_2)$
12:   $[\mathbf{g}^s_1, \mathbf{g}^l_1] = \mathrm{GMMs}(\mathbf{T}[*, j_1])$
13:   $[\mathbf{g}^s_2, \mathbf{g}^l_2] = \mathrm{GMMs}(\mathbf{T}[*, j_2])$
14:   $\mathbf{g}^s = \mathbf{g}^s_1 \cap \mathbf{g}^s_2$
15:   $\mathbf{c}^e = \mathrm{Choose}(\mathbf{g}^s, \complement_{[r]}\mathbf{g}^s)$
16:   $\mathbf{c}^d = \complement_{[r]}\mathbf{c}^e$
17:   $\mathbf{T}[I_{\mathbf{c}^d}, P_1 : P_2 - 1] = \emptyset$
18:   $\mathbf{T}[I_{\mathbf{c}^e}, c - (P_2 - P_1) : c] = \emptyset$
19:   $j = P_1$
20: **end while**
21: $\mathbf{T}_{align} = \mathbf{T}$

It is worth noting that Algorithm 1 has the capability to fix clustering errors. Recall in Equation 5, the smaller cluster $\mathbf{g}^s$ is obtained by the intersection operation $\mathbf{g}^s = \mathbf{g}^s_1 \cap \mathbf{g}^s_2 = \mathbf{c}^s \cup (\mathbf{c}^l_{\text{scatter}} \cap \mathbf{c}^{l'}_{\text{scatter}})$, which is an ideal case as claimed. Unfortunately in practice, two cases will cause errors. On the one hand, $\mathbf{g}^s_1$ and $\mathbf{g}^s_2$ fail to cover all the points belonging to the smaller cluster $\mathbf{c}^s$ and thus some points $\mathbf{c}^s_{g^l}$ will be incorrectly assigned to $\mathbf{g}^l$. We denote it as "case A". On the other hand, a set of points $\mathbf{c}^l_{g^s}$ remain after the intersection of $\mathbf{c}^l_{\text{scatter}} \cap \mathbf{c}^{l'}_{\text{scatter}}$, which indicates that some points of larger cluster $\mathbf{c}^l$ will be wrongly classified as $\mathbf{g}^s$. We call it as "case B". Since the smaller cluster $\mathbf{c}^s$ may corresponds to the encryption operations or relates to the delay instructions (as indicated in line 15 of Algorithm 1), there will totally be two error situations in Algorithm 1, that is $\mathbf{c}^e_{g^d}$ and $\mathbf{c}^g_{g^e}$, as summarized in Table III. The $\mathbf{c}^e_{g^d}$ indicates that the samples of $\mathbf{c}^e$ (involving encryption operations) are incorrectly assigned into the group $\mathbf{c}^d$. While $\mathbf{c}^d_{g^e}$ implies that the samples of $\mathbf{c}^d$ (involving delay operations) are mistakenly classified into the group $\mathbf{c}^e$. The former error cannot be fixed since one waveform segment with length $(P_2 - P_1)$ of $\mathbf{c}^d$ will be discarded after the identification of clusters, as shown in line 17. Hence for the measurements $\mathbf{T}[I_{\mathbf{c}^e_{g^d}}, *]$, it indicates that one segment of normal encryption operations is eliminated and thus cannot be synchronized anymore. However considering the latter case, there is still a chance to make amends. Since the points $\mathbf{c}^d_{g^e}$ have been clustered into $\mathbf{c}^e$, it remains the waveform segment (that should be removed) of $\mathbf{T}[I_{\mathbf{c}^d_{g^e}}, P_1 : (P_2 - 1)]$, which is one length delay and thus consistent with the segment of $\mathbf{c}^d$ to be eliminated in the next clustering. Hence it still stands a good possibility to be classified back to $\mathbf{c}^d$ in the next vertical clustering and the related segment can be eliminated. Therefore, the latter error situation holds promise to be fixed during the procedure of VCAlign with GMMs.

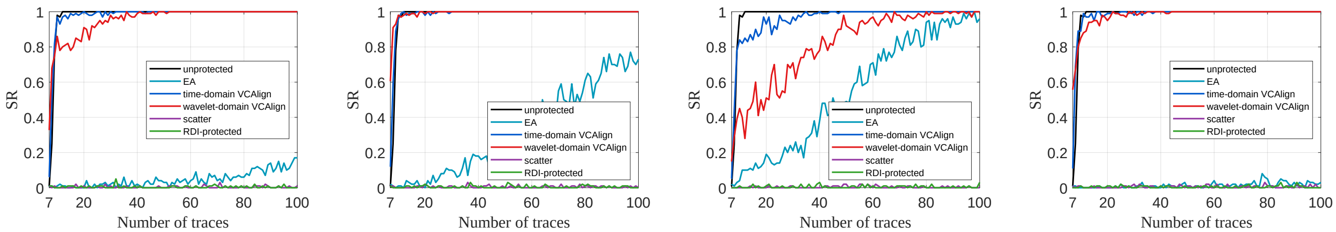
It can be indicated in Algorithm 1 that this paradigm further reduces the computational overhead. Precisely, it only needs 2 (rather than n) clustering operations for one loop of delay with n as its largest length. Note that VCAlign is a generic approach that can be enhanced by incorporating specific clustering algorithms and strategies. Our goal in this section is to provide a feasible and practical paradigm for VCAlign.

V. COMPARISON AND EXPERIMENTAL RESULTS

In this section, we assess the alignment performance of VCAlign with GMMs clustering method, in comparison with two established methods: the widely used *Elastic Alignment* (EA) and the novel *scatter* method, both of which have been demonstrated to effectively address alignment challenges.

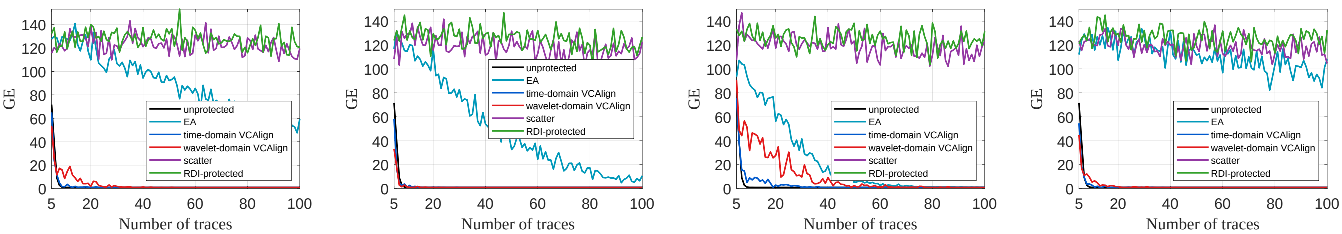
A. Experimental Setup

We choose AES-128 as encryption algorithm since it is a commonly used symmetric block cipher. We exploit four types of RDI (varying with different generation methods as introduced in Section I-B) to protect AES-128, which are summarized in Table I. RDI is implemented as in [12, Appendix B] and [13, Appendix B], and the implementation code is available in Github [35]. Our attack target is the output of first-byte SubByte operation in the first AES round. At the beginning, only one delay is inserted prior to the target operation. Recall that VCAlign achieves the alignment by eliminating delays one by one, hence we provide the elimination of one delay as an example to showcase the effectiveness of VCAlign. Subsequently, we present experimental results under more realistic condition involving multiple delays prior to the target operation, further validating the effectiveness of VCAlign. We utilize Chipwhisperer-Pro (Level 3 Starter Kit) to capture the power consumption of protected AES-128 running on the 32-bit ARM Cortex-M4 micro-controller, with traces subsequently analyzed on a PC equipped with an Intel i7-9700 CPU. We should highlight that we do not select the open dataset of FM [36] as target since its measurements have



(a) RDI-PU (b) RDI-BT (c) RDI-FM (d) RDI-IFM

Fig. 9. SR results for RDI-protected implementations before and after alignment. The term "unprotected" indicates an unprotected AES-128 implementation that is set as the baseline.



(a) RDI-PU (b) RDI-BT (c) RDI-FM (d) RDI-IFM

Fig. 10. GE results for RDI-protected implementations before and after alignment.

TABLE IV
SNR COMPARISON ON FOUR IMPLEMENTATIONS PROTECTED BY RDI

	RDI-PU	RDI-BT	RDI-FM	RDI-IFM
Original	0.02	0.03	0.01	0.01
EA	0.18	0.43	0.80	0.06
time-domain VCAlign	8.59	12.59	4.35	15.79
wavelet-domain VCAlign	3.81	11.17	0.86	9.48

been manually processed which do not meet attack requirement (beginning before the insertion of an individual delay). And since ASCAD [19] causes the random displacement by manually discarding points of encryption operations randomly, instead of inserting random delays, it is excluded in our experiments as well.

B. Experimental Results

This section begins with experimental results of removing one delay across four RDI types to demonstrate VCAlign's effectiveness and advantages. Initially, *Signal-to-Noise Rate* (SNR) is exploited to demonstrate the improvement in measurement alignment. We select EA[3] for comparison since it is a widely applied alignment method in practice. More importantly, it is non-profiled and requires no prior knowledge, which is consistent with our approach. Note that no pre-processing methods to remove small jitters are conducted before VCAlign and EA.

To begin with, Table IV provides maximum SNR of those four protected implementations before and after alignment by both of EA and VCAlign with GMMs. Note that both of time-domain sample and wavelet-domain coefficients ($\mathbf{v}_w = [cA_1, cD_1]$ with wavelet basis function as Db2) are leveraged as the feature vectors for VCAlign. It can be indicated in Table IV that VCAlign (either in time domain or in wavelet domain) outperforms EA method for all the RDI-protected cases and substantially improves the SNR of the protected implementations. In particular, for RDI-IFM, the SNR of traces aligned by time-domain VCAlign is approximately 263 (1579) times greater than that of traces aligned by EA (resp., original traces), demonstrating the significant enhancement achieved by our proposed alignment method. To exhibit the advanced alignment performance of VCAlign, we also illustrate the waveform pattern of three exemplary traces (respectively from $\mathbf{T}^1$, $\mathbf{T}^2$ and $\mathbf{T}^3$) before and after alignment in Appendix. It is shown in all figures of Appendix that, EA method seems to synchronize the target point but fails to align the whole waveform pattern[4], while VCAlign with GMMs nearly perfectly eliminates the delays and the whole waveform pattern (before the insertion of D^2) related to encryption operations are well synchronized whether in time or wavelet domain.

Now we utilize *Success Rate* (SR) and *Guessing Entropy* (GE) [37] to depict attack efficiency after the removal of random delays. Note that CPA is utilized after the alignment of both EA and VCAlign. To further demonstrate the effectiveness and advantage of VCAlign, the novel *scatter* method—which has been shown to effectively address misalignment [27]—is included as an additional comparison. SR and GE results for those four RDI cases are depicted in Figure 9 and Figure 10, respectively. We also provide the SR and GE results of attacking an unprotected AES-128 implementation and implementations protected by various RDI methods without any alignment by CPA as baselines. It is indicated in both Figure 9 and Figure 10 that VCAlign with GMMs outperforms EA for all our experimental settings in terms of attack effectiveness, which is consistent with the SNR results.

[3]We use the official Insecptor 2022.2.1 to perform EA on protected implementations.

[4]This may due to the fact that EA chooses not to dispose of samples, but to merely compress or stretch areas, decreasing the possibility of removing interesting information [25].

Regarding the scatter method, the number of traces required for a successful attack is $1,701$, $1,804$, $1,606$ and $2,602$ for RDI-PU, RDI-BT, RDI-FM and RDI-IFM, respectively. While as shown in both Figure 9 and Figure 10, both time-domain and wavelet-domain VCAlign successfully retrieves the key byte using fewer than 100 traces, demonstrating substantially higher attack efficiency compared to the scatter method. Especially, in the case of RDI-IFM, the state-of-the-art generation method for RDI, both the time-domain and wavelet-domain VCAlign methods are capable of successfully attacking within 50 traces. In particular, the attack ability of time-domain VCAlign is nearly equivalent to that of CPA when attacking unprotected implementations, as demonstrated in Figure 9d, which suggests that delays are almost perfectly eliminated by time-domain VCAlign. In contrast, the EA and scatter methods only decrease GE results to around 100 within 100 traces, indicating the need for a significantly larger number of measurements to achieve successful attacks. Compared with the original RDI-protected implementation, the attack ability is also substantially improved by leveraging VCAlign, since the SR and GE results of time-domain VCAlign are even comparable with those of unprotected implementation in the cases of RDI-PU, RDI-BT and RDI-IFM. It is worth noting that these traces have not undergone pre-processing for jitter removal, indicating the effectiveness of VCAlign in scenarios involving small jitters.

Remark 1. *Indicated by both SNR (in Table IV) and GE results (in Figure 10), time-domain VCAlign seems to be more powerful than wavelet-domain VCAlign. This may due to the fact that the wavelet basis function and the level for wavelet decomposition selected in this study are not optimal for the traces in our setting. However, developing strategies for selecting an appropriate wavelet basis function and decomposition level to improve VCAlign will be addressed in future work.*

C. Removing Multiple Delays

In this section, we evaluate the effectiveness of VCAlign by inserting multiple random delays prior to the target operation—a scenario that more closely reflects real-world attack conditions. The countermeasure is implemented following [12], [13], with 10 random delays inserted per AES round: before AddRoundKey, four during SubBytes+ShiftRows, before each MixColumns, and after MixColumns. To further increase delays prior to the target operation—the first-byte Sbox output in the first AES round—an additional dummy AES round incorporating random delays is inserted at the beginning. Since EA has been shown to outperform the scatter method in Section V-B and is a widely adopted alignment technique, the following comparison focuses exclusively on VCAlign and EA. In addition, it should be noted that IFM is a more advanced countermeasure than PU, BT and FM, and as demonstrated in Figure 9 and Figure 10, attacking IFM using EA is more challenging than attacking other three types [5]. Therefore, this section presents experimental results only for RDI-IFM. Since the selection of wavelet parameters—which falls outside the scope of this paper—is critical to the performance of wavelet-domain VCAlign, only the time-domain variant of VCAlign is considered in this sub-section.

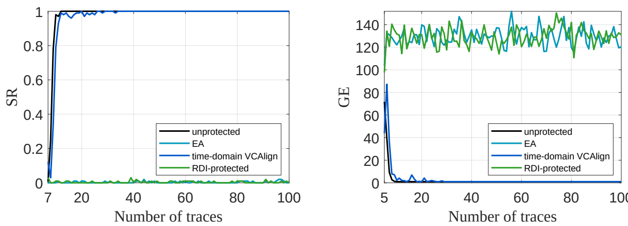
The SNR of the original protected implementation is 0.0204. After aligning the traces using time-domain VCAlign with GMMs and EA, the SNR increases to 40.81 and 0.0229, respectively. This represents an improvement by a factor of approximately 2000× with VCAlign, whereas EA yields only a marginal gain. The limited enhancement from EA can be attributed to two main factors: firstly, as indicated in Table IV, EA exhibits inherently poor alignment performance against RDI-IFM; secondly, the insertion of multiple random delays further exacerbates the difficulty of effective alignment for EA. In contrast, VCAlign achieves synchronization by eliminating each delay individually. Although imperfect removal of preceding delays may introduce additional noise, the method remains largely robust to the presence of multiple delays. This underscores the superior capability of VCAlign in handling complex, real-world countermeasures and in significantly recovering signal quality.

In terms of computational complexity, both alignment methods were performed on the experimental platform as introduced in Section V-A, processing a total of $40,000$ power traces with each comprising $25,000$ sample points. EA method required approximately 18.91 hours to complete full alignment of the dataset. In contrast, time-domain VCAlign using GMMs to classify and remove individual delays sequentially—completed the removal of each delay in approximately 3 minutes, resulting in a total alignment time of merely 36 minutes for the entire set of traces. This remarkable reduction in processing time—by a factor of about 31.5 compared to EA—demonstrates not only VCAlign's superior alignment effectiveness as established previously, but also its substantially higher computational efficiency, underscoring its strong practicality for large-scale side-channel analysis tasks.

Figure 11 showcases the SR and GE results of RDI-IFM protected implementations with multiple delays before and after alignment. It is demonstrated that the attack results are consistent with the SNR findings. Notably, EA fails to achieve a successful attack within 100 traces, performing comparably to the protected implementation without any alignment. In contrast, after alignment by time-domain VCAlign with GMMs, the CPA attack results closely resemble those of the unprotected implementation. This indicates that VCAlign effectively removes random and multiple delays, restoring signal alignment nearly to the level of the original unprotected power traces. These results highlight VCAlign's exceptional alignment quality and its strong practical advantage in enabling efficient and successful side-channel attacks.

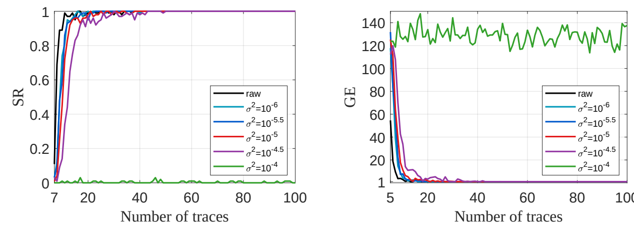
In conclusion, our practical experiments—evaluating implementations protected by both individual (Section V-B) and multiple delays (Section V-C)—demonstrate that VCAlign, combined with GMM clustering, has a powerful ability to eliminate the delays effectively and achieves high-quality trace alignment. The method significantly outperforms both EA and scatter method in terms of both alignment performance and attack effectiveness. Furthermore, as discussed in Sec-

[5]VCAlign finds FM more difficult to attack, as FM has a larger number of groups, which reduces the number of traces per group. This reduction hinders classification performance of VCAlign under a fixed total number of traces.



(a) SR results (b) GE results

Fig. 11. SR and GE results for RDI-IFM protected implementations with multiple delays before and after alignment. The term "unprotected" indicates an unprotected AES-128 implementation that is set as the baseline.



(a) SR results (b) GE results

Fig. 12. SR and GE results for noise-injected RDI-IFM with one delay.

tion V-C, VCAlign exhibits substantially lower computational complexity compared to EA. Its consistent performance across both time and wavelet domains further confirms the generality and practicality of the VCAlign approach. Verified through extensive practical experiments, VCAlign can be regarded as an advanced and efficient solution for alignment in the presence of random delays.

D. Noise Tolerance

Since the performance of VCAlign depends heavily on the distinctness of side-channel leakage across instructions and the precision of binary classification—both of which are susceptible to noise—we assess its robustness under noisy conditions through a series of experiments in this section. These experiments are conducted using traces obtained from the RDI-IFM implementation protected with a single delay, which was previously evaluated in Section V-B. Specifically, independent and identically distributed (i.i.d.) Gaussian noise $N \sim \mathcal{N}(0, \sigma^2)$ is manually added to each sample point across all traces in the RDI-IFM dateset. It should be noted that the traces were acquired experimentally from physical hardware devices and thus inherently contain measurable noise. Totally 5 different noise levels including $\log_{10} \sigma^2 = \{-6, -5.5, -5, -4.5, -4\}$ are considered. Notably, when the noise variance reaches $\sigma^2 = 10^{-4}$, the amplitude of the noise reaches approximately 0.01—a magnitude comparable to the intrinsic power difference between different instructions (groups) in our experimental setup (as shown in Figure 4 and 7). Such a noise level is therefore sufficient to obscure meaningful features in the power traces, presenting a substantial challenge for alignment.

Figure 12 presents the CPA results after alignment using time-domain VCAlign with GMMs for those 5 noise-injected trace sets described above. For comparison, the attack results on the original RDI-IFM traces (labeled as "raw") are also included as a baseline. It can be observed that from the unmodified case up to a noise variance of $\sigma^2 = 10^{-4.5}$, the alignment performance of VCAlign gradually declines—reflected in a progressively more challenging key recovery process—yet the difference remains extremely small. Throughout this range, the correct key is consistently recovered within less than 60 traces. However, when the noise variance reaches $\sigma^2 = 10^{-4}$, a sharp transition in attack difficulty occurs: even when using 100 traces, GE shows no descending trend, indicating a substantial loss of alignment effectiveness under severe noise conditions.

Therefore, the noise tolerance of VCAlign is intrinsically linked to the magnitude of intrinsic leakage differences between instructions. The experimental results in this section indicate that VCAlign's alignment efficacy diminishes significantly when the noise reaches a level comparable to the inherent inter-instruction leakage amplitude—as observed at a noise variance $\sigma^2 = 10^{-4}$ in our experimental setup. Conversely, when the noise remains below this critical threshold, VCAlign maintains robust performance with only slight degradation in alignment quality. These findings underscore that the practical applicability of VCAlign is therefore determined by the ratio between the noise amplitude and the inter-instruction leakage difference, rather than by any fixed noise level.

VI. Discussion and Conclusion

A. Discussion of Countermeasures

To counter VCAlign, two strategic directions can be considered to disrupt its alignment mechanism. First, since VCAlign relies on initially synchronized traces to function effectively, deliberately introducing a random delay before the start of the encryption process could disrupt this synchronization. By varying the starting point of the computation pseudo-randomly across different executions, VCAlign would face increased difficulty in identifying consistent alignment anchors, thereby reducing the effectiveness of trace alignment. Second, current strong performance of VCAlign can be attributed to the structured and repetitive design of the inserted delays. Each delay segment—corresponding to a loop iteration—has uniform length, resulting in consistent and predictable leakage patterns. This regularity provides VCAlign with multiple candidate points within each segment for binary classification, enabling it to either select the most discriminative point or leverage information from the entire segment to improve alignment accuracy. To mitigate this, a dynamic compilation strategy could be employed to insert instructions of random types and durations in a highly irregular manner. While the vertical alignment perspective of VCAlign may still apply in theory, the increased irregularity and noise would severely challenge the classification step, making precise delay identification and correction considerably more difficult.

B. Conclusion

In this paper, we provide a novel vertical perspective to give a full play to the instruction effects on physical leakage. With this new perspective, RDI exposures the existence of delays, allowing adversaries to quickly detect its presence without any prior knowledge. In addition, a generic and efficient alignment approach VCAlign is proposed, which requires no reference, no profiling stage and no prior knowledge. To prove its feasibility, we further provide a pragmatic example to feed VCAlign with GMMs clustering method. VCAlign with GMMs is verified by practical experiments that it is enhanced and powerful to eliminate random delays and synchronize the encryption-related waveform segments.

Note that the detection method is not only for RDI but also useful when applied in the countermeasures who are designed to change the instruction flows for protection. While VCAlign is devised for random delays generated with regular loops. However, the vertical perspective could still take its advantage when confronting the delays with varying instructions. For example, the points sampled at where D^1 is inserted should also have n groups with n limited by the size of instruction sets for delay generation. And those n groups are also informative for adversaries. In addition, VCAlign is capable of eliminating delays even in the presence of small jitters, as adjacent samples tend to exhibit consistent distributions due to the typically high-frequency acquisition process. Recall that traces are not pre-processed prior to the alignment by VCAlign in the practical experiments in Section V.

The strength of VCAlign is to allow an adversary without any prior knowledge to eliminate the delays effectively with no profiling stage. Furthermore, VCAlign is generic and thus can be further improved by constructing more informative feature vector and feeding with enhanced clustering strategy. The proposed novel vertical perspective is anticipated to enhance the effectiveness of attacks targeting countermeasures such as random delay insertions and bring new vitality to the application of the instruction effects on side-channel leakage.

REFERENCES

[1] P. C. Kocher, "Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems," in *Advances in Cryptology—CRYPTO'96: 16th Annual International Cryptology Conference Santa Barbara, California, USA August 18–22, 1996 Proceedings 16*. Springer, 1996, pp. 104–113.

[2] P. Kocher, J. Jaffe, and B. Jun, "Differential power analysis," in *Advances in Cryptology—CRYPTO'99: 19th Annual International Cryptology Conference Santa Barbara, California, USA, August 15–19, 1999 Proceedings 19*. Springer, 1999, pp. 388–397.

[3] D. Agrawal, B. Archambeault, J. R. Rao, and P. Rohatgi, "The em side—channel (s)," in *Cryptographic Hardware and Embedded Systems-CHES 2002: 4th International Workshop Redwood Shores, CA, USA, August 13–15, 2002 Revised Papers 4*. Springer, 2003, pp. 29–45.

[4] S. Mangard, E. Oswald, and T. Popp, *Power analysis attacks: Revealing the secrets of smart cards*. Springer Science & Business Media, 2008, vol. 31.

[5] C. Clavier, J.-S. Coron, and N. Dabbous, "Differential power analysis in the presence of hardware countermeasures," in *Cryptographic Hardware and Embedded Systems—CHES 2000: Second International Workshop Worcester, MA, USA, August 17–18, 2000 Proceedings 2*. Springer, 2000, pp. 252–263.

[6] F. Zhang, X. Dong, B. Yang, Y. Zhou, and K. Ren, "A systematic evaluation of wavelet-based attack framework on random delay countermeasures," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 1407–1422, 2019.

[7] M. Bucci, R. Luzzi, M. Guglielmo, and A. Trifiletti, "A countermeasure against differential power analysis based on random delay insertion," in *2005 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2005, pp. 3547–3550.

[8] Y. Zafar, J. Park, and D. Har, "Random clocking induced dpa attack immunity in fpgas," in *2010 IEEE International Conference on Industrial Technology*. IEEE, 2010, pp. 1068–1070.

[9] S. Mangard, "Hardware countermeasures against dpa–a statistical analysis of their effectiveness," in *Topics in Cryptology–CT-RSA 2004: The Cryptographers' Track at the RSA Conference 2004, San Francisco, CA, USA, February 23-27, 2004, Proceedings*. Springer, 2004, pp. 222–235.

[10] D. Couroussé, T. Barry, B. Robisson, P. Jaillon, O. Potin, and J.-L. Lanet, "Runtime code polymorphism as a protection against side channel attacks," in *Information Security Theory and Practice: 10th IFIP WG 11.2 International Conference, WISTP 2016, Heraklion, Crete, Greece, September 26–27, 2016, Proceedings*. Springer, 2016, pp. 136–152.

[11] M. Tunstall and O. Benoit, "Efficient use of random delays in embedded software," in *Information Security Theory and Practices. Smart Cards, Mobile and Ubiquitous Computing Systems: First IFIP TC6/WG 8.8/WG 11.2 International Workshop, WISTP 2007, Heraklion, Crete, Greece, May 9-11, 2007. Proceedings 1*. Springer, 2007, pp. 27–38.

[12] J.-S. Coron and I. Kizhvatov, "An efficient method for random delay generation in embedded software," in *Cryptographic Hardware and Embedded Systems-CHES 2009: 11th International Workshop Lausanne, Switzerland, September 6-9, 2009 Proceedings*. Springer, 2009, pp. 156–170.

[13] ——, "Analysis and improvement of the random delay countermeasure of ches 2009," in *Cryptographic Hardware and Embedded Systems, CHES 2010: 12th International Workshop, Santa Barbara, USA, August 17-20, 2010. Proceedings 12*. Springer, 2010, pp. 95–109.

[14] S. Nagashima, N. Homma, Y. Imai, T. Aoki, and A. Satoh, "DPA using phase-based waveform matching against random-delay countermeasure," in *International Symposium on Circuits and Systems (ISCAS 2007), 27-20 May 2007, New Orleans, Louisiana, USA*. IEEE, 2007, pp. 1807–1810. [Online]. Available: https://doi.org/10.1109/ISCAS.2007.378024

[15] F. Durvaux, M. Renauld, F.-X. Standaert, L. Van Oldeneel Tot Oldenzeel, and N. Veyrat-Charvillon, "Efficient removal of random delays from embedded software implementations using hidden markov models," in *Smart Card Research and Advanced Applications: 11th International Conference, CARDIS 2012, Graz, Austria, November 28-30, 2012, Revised Selected Papers 11*. Springer, 2013, pp. 123–140.

[16] E. Cagli, C. Dumas, and E. Prouff, "Convolutional neural networks with data augmentation against jitter-based countermeasures: Profiling attacks without pre-processing," in *Cryptographic Hardware and Embedded Systems–CHES 2017: 19th International Conference, Taipei, Taiwan, September 25-28, 2017, Proceedings*. Springer, 2017, pp. 45–68.

[17] J. Kim, S. Picek, A. Heuser, S. Bhasin, and A. Hanjalic, "Make some noise. unleashing the power of convolutional neural networks for profiled side-channel analysis," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 148–179, 2019.

[18] B. Timon, "Non-profiled deep learning-based side-channel attacks with sensitivity analysis," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 107–131, 2019.

[19] R. Benadjila, E. Prouff, R. Strullu, E. Cagli, and C. Dumas, "Deep learning for side-channel analysis and introduction to ascad database," *Journal of Cryptographic Engineering*, vol. 10, no. 2, pp. 163–188, 2020.

[20] Y.-S. Won, X. Hou, D. Jap, J. Breier, and S. Bhasin, "Back to the basics: Seamless integration of side-channel pre-processing in deep neural networks," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 3215–3227, 2021.

[21] D. Strobel and C. Paar, "An efficient method for eliminating random delays in power traces of embedded software," in *Information Security and Cryptology-ICISC 2011: 14th International Conference, Seoul, Korea, November 30-December 2, 2011. Revised Selected Papers 14*. Springer, 2012, pp. 48–60.

[22] C. H. Gebotys, C. C. Tiu, and X. Chen, "A countermeasure for em attack of a wireless pda," in *International Conference on Information Technology: Coding and Computing (ITCC'05)-Volume II*, vol. 1. IEEE, 2005, pp. 544–549.

[23] O. Schimmel, P. Duplys, E. Boehl, J. Hayek, R. Bosch, and W. Rosenstiel, "Correlation power analysis in frequency domain," in *COSADE 2010 First International Workshop on Constructive SideChannel Analysis and Secure Design*, 2010.

[24] Y. Lu, K. Boey, M. O'Neill, J. V. McCanny, and A. Satoh, "Is the differential frequency-based attack effective against random delay insertion?" in *Proceedings of the IEEE Workshop on Signal Processing Systems, SiPS 2009, October 7-9, 2009, Tampere, Finland*. IEEE, 2009, pp. 051–056. [Online]. Available: https://doi.org/10.1109/SIPS.2009.5336291

[25] J. G. van Woudenberg, M. F. Witteman, and B. Bakker, "Improving differential power analysis by elastic alignment," in *Topics in Cryptology–CT-RSA 2011: The Cryptographers' Track at the RSA Conference 2011,*

San Francisco, CA, USA, February 14-18, 2011. Proceedings. Springer, 2011, pp. 104–119.

[26] S. Salvador and P. Chan, “Toward accurate dynamic time warping in linear time and space,” *Intelligent Data Analysis*, vol. 11, no. 5, pp. 561–580, 2007.

[27] H. Thiebeauld, G. Gagnerot, A. Wurcker, and C. Clavier, “SCATTER: A new dimension in side-channel,” in *Constructive Side-Channel Analysis and Secure Design - 9th International Workshop, COSADE 2018, Singapore, April 23-24, 2018, Proceedings*, ser. Lecture Notes in Computer Science, J. Fan and B. Gierlichs, Eds., vol. 10815. Springer, 2018, pp. 135–152. [Online]. Available: https://doi.org/10.1007/978-3-319-89641-0_8

[28] X. Charvet and H. Pelletier, “Improving the dpa attack using wavelet transform,” in *NIST Physical Security Testing Workshop*, vol. 46, 2005.

[29] R. A. Muijrers, J. G. van Woudenberg, and L. Batina, “Ram: Rapid alignment method,” in *Smart Card Research and Advanced Applications: 10th IFIP WG 8.8/11.2 International Conference, CARDIS 2011, Leuven, Belgium, September 14-16, 2011, Revised Selected Papers 10*. Springer, 2011, pp. 266–282.

[30] K. M. Abdellatif, D. Couroussé, O. Potin, and P. Jaillon, “Filtering-based cpa: a successful side-channel attack against desynchronization countermeasures,” in *Proceedings of the Fourth Workshop on Cryptography and Security in Computing Systems*, 2017, pp. 29–32.

[31] F. Zhang, X. Dong, B. Yang, Y. Zhou, and K. Ren, “A systematic evaluation of wavelet-based attack framework on random delay countermeasures,” *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 1407–1422, 2020. [Online]. Available: https://doi.org/10.1109/TIFS.2019.2941774

[32] Q. Wu, F. Zhang, S. Guo, K. Yang, and H. Shen, “A unified and fully automated framework for wavelet-based attacks on random delay,” *IEEE Trans. Computers*, vol. 73, no. 9, pp. 2206–2219, 2024. [Online]. Available: https://doi.org/10.1109/TC.2024.3416682

[33] M. Misiti, Y. Misiti, G. Oppenheim, and J.-M. Poggi, “Wavelet toolbox user’s guide,” *The MathWorks Inc., Natick, MA*, vol. 15, pp. 30–35, 1996.

[34] V. A. Epanechnikov, “Non-parametric estimation of a multivariate probability density,” *Theory of Probability & Its Applications*, vol. 14, no. 1, pp. 153–158, 1969.

[35] Qianmei Wu, “Implementation of random delay insertion,” https://github.com/WuXiMei/RDI-implemented-by-ARM-assembly.git, 2023.

[36] BKizhvatov. (2021) randomdelays-traces. [Online]. Available: https://github.com/ikizhvatov/randomdelays-traces

[37] F.-X. Standaert, T. G. Malkin, and M. Yung, “A unified framework for the analysis of side-channel key recovery attacks,” in *Advances in Cryptology-EUROCRYPT 2009: 28th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cologne, Germany, April 26-30, 2009. Proceedings 28*. Springer, 2009, pp. 443–461.

Qianmei Wu was born in 1999. She received the bachelor’s degree from the College of Computer Science and Technology, Jilin University, in 2020. She is currently pursuing the Ph.D. degree with the College of Computer Science and Technology, Zhejiang University. Her research interests include hardware security, cryptography, and side-channel analysis.

Chengdong Xie was born in 1998. He received the bachelor’s degree from the College of Mathematics, Sichuan University, in 2021. He is currently pursuing the Ph.D. degree with the College of Mathematics, Sichuan University. His research interests include algebraic topology, higher category theory and science of complexity.

Wei Cheng received the Ph.D. degree in information and communications from Télécom Paris, Institut Polytechnique de Paris, France, in 2021. He is currently a Post-Doctoral Researcher at Secure-IC SAS, and also an (invited) Associate Researcher at Télécom Paris. His research interests include information theory, side-channel analysis, related countermeasures (mainly on code-based masking, including inner product masking, direct sum masking, polynomial masking, and other variants) for embedded systems, and secure cryptographic implementations. He also works on machine learning-based analysis on physical unclonable functions (PUFs).

Fan Zhang was born in 1978. He received the Ph.D. degree from the Department of Computer Science and Engineering, University of Connecticut, USA, in 2012. He is currently a full professor in the School of Cyber Science and Technology, College of Computer Science and Technology, Zhejiang University. His research interests include system security, hardware security, network security, cryptography and computer architecture.